

# Workshop 1: GitHub & NumPy

## TECH2: Introduction to Programming, Data, and Information Technology

Richard Foltyn

*NHH Norwegian School of Economics*

October 2, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/TECH2-H26>

### Exercise 1: Summing lists and arrays

In this exercise, we investigate another difference between built-in lists and NumPy arrays: performance. We do this by comparing the execution time of different implementations of the `sum()` function.

1. Create a list `lst` and a NumPy array `arr`, each of them containing the sequence of ten values 0, 1, 2, ..., 9.

*Hint:* You can use the list constructor `list()` and combine it with the `range()` function, which returns an object representing a range of integers.

*Hint:* You should create the NumPy array using `np.arange()`.

2. We want to compute the sum of integers contained in `lst` and `arr`. Use the built-in function `sum()` to sum elements of a list. For the NumPy array, use the NumPy function `np.sum()`.
3. You are interested in benchmarking which summing function is faster. Repeat the steps from above, but use the cell magic `%timeit` to time the execution of a statement as follows:

```
%timeit statement
```

4. Recreate the list and array to contain 100 integers starting from 0, and rerun the benchmark.
5. Recreate the list and array to contain 10,000 integers starting from 0, and rerun the benchmark.

What do you conclude about the relative performance of built-in lists vs. NumPy arrays?

### Exercise 2: Maximizing quadratic utility

Assume that an individual derives utility from consuming  $c$  items according to the following utility function  $u(\bullet)$ :

$$u(c) = -A(c - B)^2 + C$$

where  $A > 0$ ,  $B > 0$ , and  $C$  are parameters, and  $c$  is the consumption level.

In this exercise, you are asked to locate the consumption level that delivers the maximum utility.

1. Define a function called `util()` which takes as arguments the consumption level  $c$  and three additional arguments  $A$ ,  $B$ , and  $C$ , which are the parameters of  $u(\bullet)$  defined above. The function should return the utility associated with the given consumption level  $c$ .

2. Write a function `find_max_cons()` which takes as arguments a sequence of candidate consumption levels and the three parameters  $A$ ,  $B$ , and  $C$ , and returns the maximum utility as well as the consumption level at which utility is maximized. The function definition should look like this:

```
def find_max_cons(candidates, A, B, C):  
    """  
    Find the consumption level that maximizes utility from a  
    list of candidates.  
  
    Parameters  
    -----  
    candidates : list or array-like  
        List of candidate consumption levels to evaluate.  
    A, B, C : float  
        Parameters of the utility function.  
  
    Returns  
    -----  
    u_max  
        Maximized utility  
    cons_max  
        Consumption at which utility is maximized  
    """
```

Your algorithm should perform the following steps:

1. Define the variable `u_max = -np.inf` (negative infinity) as the initial value.
  2. Loop through all candidate consumption levels, and compute the associated utility. If this utility is larger than the previous maximum value `u_max`, update `u_max` and store the associated consumption level `cons_max`.
  3. Return `u_max` and `cons_max` after the loop terminates.
3. Find the maximum:
    1. Create an array `cons` of 51 candidate consumption levels which are uniformly spaced on the interval  $[0, 4]$ .  
*Hint:* Use `np.linspace()` for this task.
    2. Use the parameters  $A = 1$ ,  $B = 2$ , and  $C = 10$ .
    3. Use the function `find_max_cons()` to compute the maximum utility and the associated optimal consumption level, and print the results.
  4. Repeat the exercise, but instead use vectorized operations from NumPy:
    1. Compute and store the utility levels for *all* elements in `cons` at once (simply apply the formula to the whole array).
    2. Locate the index of the maximum utility level using `np.argmax()`.
    3. Use the index returned by `np.argmax()` to retrieve the maximum utility and the corresponding consumption level, and print the results.